



A PROJECT REPORT ON

SOLVEXA

*A Smart Civic Issue Reporting and Resolution
Platform*

A web application that connects citizens, administrators
and field staff to report, assign, track and resolve
community problems.

Submitted by

Dontha Pranathi

Technologies: React · Node.js · Express · MongoDB

September 2026

github.com/pranathisubbarao-svg/SOLVEXA

Acknowledgements

I would like to express my sincere gratitude to everyone who supported me in the successful completion of the project **SOLVEXA – A Smart Civic Issue Reporting and Resolution Platform**.

First and foremost, I am deeply thankful to my project guide for the valuable guidance, constant encouragement and constructive suggestions throughout the development of this project. Their insights helped me shape the idea into a complete, working system.

I extend my heartfelt thanks to the Head of the Department and all the faculty members of my department for providing the knowledge, facilities and a supportive learning environment that made this work possible.

I am grateful to the open-source communities behind React, Node.js, Express, MongoDB and the many libraries used in this project. Their excellent documentation and freely available tools were essential to building SOLVEXA.

I also thank my friends for their feedback while testing the application, and for sharing their own experiences of reporting civic problems, which helped me understand what users really need.

Finally, I am forever grateful to my parents and family for their love, patience and encouragement, which kept me motivated from the beginning to the end of this project.

Dontha Pranathi

Abstract

Civic problems such as potholes, broken street lights, overflowing garbage, water leaks and blocked drains affect the daily lives of millions of people, yet reporting them is often slow, and citizens rarely know whether their complaints are being addressed. Authorities, in turn, receive complaints through scattered channels, making it hard to prioritise, assign and track work.

SOLVEXA is a web-based civic issue reporting and resolution platform that brings citizens, administrators and field staff together in one transparent workflow. Citizens register with e-mail or a Google account and report an issue with its category, priority, description and location in under a minute. Each report receives a unique complaint number and moves through six clearly visible stages — Submitted, Under Review, Assigned, In Progress, Resolved and Closed. Administrators use a dedicated console with live statistics and charts to review, prioritise and assign reports to approved staff, whose own workspace lists only their assigned work. Field staff are recruited through a verified online application that validates Aadhaar and PAN numbers, collects education details and documents, and notifies applicants of the decision by e-mail.

The system is built on the MERN stack — MongoDB, Express, React and Node.js — using a three-tier architecture and a REST API. Security is ensured through bcrypt password hashing, JSON Web Token (JWT) role-based access control, verified Google sign-in, masked identity numbers and private document storage. Testing confirmed correct validation, access control and end-to-end complaint resolution. SOLVEXA demonstrates how modern web technologies can make civic grievance redressal faster, more accountable and more citizen-friendly.

Keywords: civic issue reporting, grievance redressal, MERN stack, React, Node.js, Express, MongoDB, REST API, JWT authentication, role-based access control, smart city.

Table of Contents

Acknowledgements	i
Abstract	ii
Table of Contents	iii
1. Introduction	1
1.1 Problem Statement	1
1.2 Motivation & Statement of Past/Related Work	1
1.3 Objective of the Project & Statement of Challenges	2
1.4 Product Scope	2
1.5 Uses and Benefits of the Project	3
1.6 Organisation of the Report	3
2. Background	4
2.1 Civic Grievance Redressal in India	4
2.2 Limitations of Existing Approaches	4
2.3 The Web Application Approach	4
2.4 Key Concepts and Terminology	5
3. Overall Description	6
3.1 Product Perspective	6
3.2 Product Functions	6
3.3 User Classes and Characteristics	6
3.4 Operating Environment	7
3.5 Design and Implementation Constraints	7
3.6 User Documentation	7
3.7 Assumptions and Dependencies	7
4. Project Activities	9
4.1 Requirement Gathering and Analysis	9
4.2 System Design	9
4.3 Implementation Modules	9
4.4 Testing and Verification	10
4.5 Version Control and Deployment Preparation	11
4.6 Summary of Deliverables	11

5. Data Collection	12
5.1 Data Collected from Citizens	12
5.2 Data Collected from Staff Applicants	12
5.3 System-Generated Data	13
5.4 Validation Rules	13
5.5 Data Privacy and Protection	13
5.6 Public (Anonymous) Statistics	14
6. Project Management	15
6.1 Development Methodology	15
6.2 Project Phases and Schedule	15
6.3 Tools Used for Managing the Project	15
6.4 Risk Management	16
6.5 Configuration and Secrets Management	16
7. System Architecture	17
7.1 Architectural Overview	17
7.2 Data Flow Diagram – Level 0	18
7.3 Data Flow Diagram – Level 1	19
7.4 Use Case Diagram	20
7.5 Complaint Lifecycle	20
7.6 Database Schema	21
7.7 REST API Design	22
7.8 Security Architecture	24
8. External Interface Requirements	25
8.1 User Interfaces	25
8.2 Hardware Interfaces	25
8.3 Software Interfaces	26
8.4 Communications Interfaces	26
9. System Features	27
9.1 Citizen Registration and Authentication	27
9.2 Issue Reporting and Tracking	27
9.3 Administrator Console	27
9.4 Staff Assignment and Workspace	28
9.5 Staff Recruitment and E-mail Notification	28

10. Technologies Used	29
10.1 Programming Languages	29
10.2 Frontend Technologies	29
10.3 Backend Technologies	29
10.4 Database	30
10.5 Development Tools and Services	30
11. Other Nonfunctional Requirements	31
11.1 Performance Requirements	31
11.2 Safety Requirements	31
11.3 Security Requirements	31
11.4 Software Quality Attributes	31
11.5 Business Rules	32
12. Implementation and Example Walkthrough	33
12.1 Installation and Setup	33
12.2 Running the Application	33
12.3 Example: From Report to Resolution	33
12.4 Sample API Request and Response	34
13. Conclusion and Future Enhancements	36
13.1 Conclusion	36
13.2 Limitations	36
13.3 Future Enhancements	36
14. References	37
Appendix A. Glossary	38

1. Introduction

1.1 Problem Statement

Every day, residents of towns and cities come across problems in public spaces — potholes on main roads, street lights that stay dark, overflowing garbage bins, leaking water pipes and blocked drains. These problems affect safety, health and the quality of daily life, yet reporting them is often slow and uncertain. Citizens usually do not know which department is responsible, complaints are made by phone or in person and are easily lost, and there is no simple way to find out whether anything is being done.

On the other side, the authorities who fix these problems receive complaints through many unconnected channels. Without a single system it is difficult to prioritise urgent issues, assign work to the right field staff, track progress or measure how quickly problems are being resolved. The result is duplicated effort, delays and low public trust.

SOLVEXA addresses this gap. It is a web-based civic issue reporting and resolution platform that connects three groups — **citizens** who report problems, **administrators** who review and assign them, and **field staff** who resolve them — through one transparent, trackable workflow.

1.2 Motivation & Statement of Past/Related Work

The motivation for SOLVEXA comes from a simple observation: most civic problems are small and fixable, but they stay unresolved because information does not reach the right person at the right time. A structured digital system can close this loop, give citizens a voice, and give authorities the data they need to act.

Several related efforts already exist. Government grievance portals such as the Centralised Public Grievance Redress and Monitoring System (CPGRAMS) allow citizens to lodge complaints with public authorities, and national programmes such as Swachh Bharat have used mobile applications for cleanliness complaints. Many municipal corporations also run helplines and their own portals. These systems proved that citizens are willing to report problems digitally.

However, many existing solutions are designed as general grievance registers. They often lack a visual, step-by-step status for the citizen, do not give field staff a dedicated workspace, and do not integrate staff recruitment and verification with the complaint workflow. SOLVEXA builds on the ideas of these systems while focusing on a modern user experience, role-based workspaces for every user type, live progress tracking and an integrated staff onboarding process.

1.3 Objective of the Project & Statement of Challenges

The main goal of SOLVEXA is to **make reporting a civic problem as easy as possible and to make its resolution fully transparent**. The specific objectives are:

1. Allow citizens to register (by e-mail or Google account) and report an issue with a title, category, priority, description and location in under a minute.
2. Give every complaint a unique number (for example *SOL-2026-0001*) and a six-stage status that the citizen can follow: Submitted, Under Review, Assigned, In Progress, Resolved and Closed.
3. Provide administrators with a console to review, prioritise and assign reports, with statistics and charts that show the overall situation at a glance.
4. Provide field staff with their own workspace that lists only the reports assigned to them, so they can start work and mark issues as resolved with notes for the citizen.
5. Recruit staff through a verified job application that collects identity (Aadhaar, PAN), education and documents, and notify applicants of the decision by e-mail.
6. Protect personal data through password hashing, token-based access control, masked identity numbers and private document storage.

Challenges. Building the system involved several challenges:

- **Role separation** — four kinds of users (citizen, staff, administrator and applicant) must each see exactly the right data and nothing more.
- **Sensitive data** — Aadhaar and PAN details and document images must be validated and stored securely.
- **Reliable validation** — identity numbers must be checked correctly (Aadhaar uses the Verhoeff checksum; PAN follows a fixed pattern) both in the browser and on the server.
- **Third-party integration** — Google sign-in and e-mail delivery depend on external services and credentials that must be configured and kept secret.
- **Usability** — the interface must work well for people with little technical experience and on both desktop and mobile screens.

1.4 Product Scope

SOLVEXA is a complete, working web application consisting of a React frontend and a Node.js/Express backend with a MongoDB database. Within its scope are:

- A public home page with live, anonymous statistics about reported and resolved issues.

- Citizen registration, login, Google sign-in, password reset, issue reporting and complaint tracking.
- An administrator console with an overview dashboard, report management, staff management and a citizen directory.
- A staff job application form, admin review of applications and documents, and e-mail notifications.
- A staff workspace for handling assigned reports.

Outside the current scope are native mobile applications, SMS notifications, payment features and integration with existing government systems; these are discussed as future enhancements in Chapter 13.

1.5 Uses and Benefits of the Project

SOLVEXA is useful to everyone involved in keeping a community in good condition:

Who benefits	How SOLVEXA helps
Citizens	Report problems quickly from any device, receive a complaint number, see exactly which stage the complaint has reached and who is handling it, and read the resolution notes.
Administrators	See every report in one place, filter by status, category, priority or assignee, assign work to staff with one click, and monitor workload and resolution rate through charts.
Field staff	Get a clear, prioritised list of assigned work with location and citizen contact, and update progress without paperwork.
Local government	Gain data on which problems occur most and how quickly they are fixed, supporting better planning and accountability.
The community	Cleaner, safer public spaces and greater trust that reported problems are taken seriously.

1.6 Organisation of the Report

Chapter 2 gives the background of civic grievance systems. Chapter 3 describes the overall product. Chapter 4 explains the project activities and Chapter 5 the data collected by the system. Chapter 6 covers project management. Chapter 7 presents the system architecture with data flow, use case, lifecycle and database diagrams. Chapters 8 to 11 state the interface, functional and non-functional requirements. Chapter 10 lists the technologies used, Chapter 12 explains how to install and run the system with a worked example, and Chapter 13 concludes the report with future enhancements.

2. Background

2.1 Civic Grievance Redressal in India

In India, civic services such as roads, street lighting, water supply, sanitation and drainage are mainly managed by urban local bodies — municipal corporations, municipalities and their ward offices. Citizens traditionally raise complaints by visiting ward offices, calling helplines or writing letters. Over the last decade, governments have introduced online grievance portals and mobile applications, and the growth of smartphones and mobile internet has made digital reporting practical for a large part of the population.

Digital systems make it possible to record every complaint, route it automatically, measure response times and publish performance data. They also allow citizens to follow up without repeated visits or calls. SOLVEXA is designed around these advantages.

2.2 Limitations of Existing Approaches

- **Lack of transparency** — citizens often receive no updates after filing a complaint.
- **Fragmented channels** — phone, paper and online complaints are handled separately.
- **No field-staff view** — work is assigned informally, making accountability difficult.
- **Poor prioritisation** — urgent issues can wait behind minor ones.
- **Little insight** — without data, authorities cannot see patterns or measure improvement.
- **Unverified workforce** — onboarding field staff is disconnected from the complaint system.

2.3 The Web Application Approach

SOLVEXA is built as a **web application** so that it runs in any modern browser on a phone, tablet or computer without installing anything. It follows the widely used **client-server** model: the browser runs a single-page application (SPA) written in React, which communicates with a REST API written in Node.js and Express. Data is stored in MongoDB, a document database whose flexible structure suits complaints and user profiles that contain nested information such as locations and application details. This combination — MongoDB, Express, React and Node.js — is commonly known as the **MERN stack**.

2.4 Key Concepts and Terminology

Term	Meaning in this project
SPA (Single-Page Application)	A web app that loads once and then updates the page with JavaScript instead of reloading, giving a fast, app-like experience.
REST API	A set of URLs (endpoints) on the server that the frontend calls using HTTP methods such as GET, POST and PUT, exchanging JSON data.
JWT (JSON Web Token)	A signed token issued at login and sent with later requests to prove who the user is and what role they have.
Hashing (bcrypt)	A one-way transformation of passwords so that the real password is never stored.
OAuth / Google Identity	A standard that lets users sign in with their Google account; SOLVEXA verifies the Google ID token on the server.
SMTP	The protocol used to send e-mail; SOLVEXA uses Gmail's SMTP server through Nodemailer.
Verhoeff checksum	The error-detecting algorithm used by the last digit of an Aadhaar number; SOLVEXA uses it to reject mistyped numbers.
ODM (Mongoose)	An Object–Document Mapper that defines schemas and validation for MongoDB collections.

3. Overall Description

3.1 Product Perspective

SOLVEXA is a new, self-contained system. It is not an extension of an existing product, but it is designed to work alongside existing municipal processes: administrators use it to receive and distribute work, and field staff use it to report progress. It depends on two external services — Google Identity Services for optional Google sign-in, and an SMTP e-mail service (Gmail) for notifications — and on a MongoDB Atlas cloud database. The architecture is described in detail in Chapter 7.

3.2 Product Functions

- User registration and login with e-mail and password, or with a Google account.
- Password reset using the registered e-mail address and phone number.
- Reporting of civic issues with category, priority, description and location.
- Automatic complaint numbering and a six-stage status workflow.
- Citizen dashboard and "My Complaints" page with a visual progress tracker.
- Administrator console: overview statistics and charts, report filtering and search, status and priority updates, assignment to staff, citizen directory and staff management.
- Staff job application with identity, education and document uploads; admin review, approval or rejection with e-mail notification.
- Staff workspace to start work on and resolve assigned reports.
- Public home page with live anonymous statistics and recent activity.

3.3 User Classes and Characteristics

User class	Characteristics	Main tasks
Citizen	General public; varied technical skill; mostly on mobile phones.	Register, report issues, track complaints.
Staff applicant	Person applying to work as field staff.	Complete the job application and upload documents.
Staff member	Approved field worker, e.g. electrician or sanitation worker.	View assigned reports, start work, mark resolved.

User class	Characteristics	Main tasks
Administrator	Trained official managing the platform; created by a secure server script.	Review and assign reports, approve staff, monitor statistics.
Visitor	Anyone viewing the public site without logging in.	Read information and live statistics on the home page.

3.4 Operating Environment

- **Client:** any modern web browser (Google Chrome, Microsoft Edge, Mozilla Firefox, Safari) on Windows, macOS, Linux, Android or iOS.
- **Server:** Node.js 24 runtime on any operating system (developed on Windows 11).
- **Database:** MongoDB Atlas (cloud-hosted MongoDB).
- **Network:** Internet connection for database, Google sign-in and e-mail.

3.5 Design and Implementation Constraints

- The system must be built with open-source web technologies (JavaScript, React, Node.js, MongoDB).
- The MongoDB Atlas cluster only accepts connections from IP addresses on its access list.
- Google sign-in only works from origins registered in Google Cloud Console (e.g. `http://localhost:5173`).
- Gmail requires an App Password (with 2-Step Verification) for SMTP access.
- Uploaded files are limited to 5 MB each and to JPG, PNG, WEBP or PDF formats.
- Secrets (database password, JWT secret, SMTP password) must be stored only in environment variables.

3.6 User Documentation

The project includes a *README* file in the source repository describing installation, configuration and running commands, and a *.env.example* file listing every required setting. Inside the application, forms include placeholder text and helpful messages, and each page explains its purpose (for example the staff login page explains how to apply for a staff job).

3.7 Assumptions and Dependencies

- Users have access to a device with a web browser and an internet connection.
- Citizens provide truthful information about issues and locations.

- Administrators verify staff documents manually before approval.
- External services (MongoDB Atlas, Google Identity Services, Gmail SMTP) remain available.
- The npm packages listed in Chapter 10 remain compatible with the Node.js version used.

4. Project Activities

The project was carried out as a sequence of activities, each producing a working part of the system that was tested before moving on. This section describes these activities.

4.1 Requirement Gathering and Analysis

The first activity was to understand the problem from the point of view of each user. Common civic problems were listed and grouped into eight categories — Roads & Potholes, Street Lights, Water Supply, Garbage & Waste, Drainage, Public Safety, Parks & Public Spaces and Other. The needs of citizens, administrators and staff were written down as user stories, for example: *"As a citizen, I want to see who is handling my complaint so that I know it is being worked on."* These stories became the functional requirements in Chapter 9.

4.2 System Design

Next, the architecture was designed: a React single-page application, an Express REST API and a MongoDB database. The data model (users and complaints), the complaint lifecycle, the API endpoints and the user-interface layouts were planned. The diagrams produced during this activity are shown in Chapter 7.

4.3 Implementation Modules

The system was implemented as the following modules:

#	Module	Description
1	Authentication	Registration, login, JWT tokens, Google sign-in, forgot password, separate admin login.
2	Complaint management	Report an issue, automatic complaint number, citizen dashboard and statistics.
3	Complaint tracking	My Complaints page with a six-stage progress tracker and resolution details.
4	Administrator console	Sidebar layout, overview charts, reports table with filters, report drawer for updates.
5	Staff assignment	Assign reports to approved staff, workload view, assignee filter, automatic status change.
6	Staff workspace	Staff-only page listing assigned work; start work, resolve with notes, reopen.

#	Module	Description
7	Staff recruitment	Five-step job application with Aadhaar/PAN validation and document upload; admin review.
8	Notifications	E-mail to applicants on approval or rejection using Nodemailer and Gmail SMTP.
9	Public website	Animated home page, live anonymous statistics, category shortcuts, FAQ.

4.4 Testing and Verification

Each module was tested as it was built. Testing combined automated browser checks (using a headless Chrome browser to click through pages and take screenshots), direct API tests and code linting with ESLint. Representative test cases are listed below.

Test case	Input / action	Expected result	Result
Register citizen	Valid name, e-mail, phone, password	Account created; redirected to login	Pass
Duplicate e-mail	Register with an existing e-mail	Error: Email already registered	Pass
Invalid Aadhaar	Number failing the Verhoeff checksum	Error: Enter a valid 12-digit Aadhaar number	Pass
Invalid PAN	ABC123	Error: Enter a valid PAN	Pass
Under-age applicant	Date of birth giving age below 18	Error: must be between 18 and 65	Pass
Wrong photo type	PDF uploaded as passport photo	Error: photo must be JPG, PNG or WEBP	Pass
Rejected upload clean-up	Any failed application	No uploaded files left on the server	Pass
Admin API without token	Call /api/admin/staff with no token	401 Please login as admin	Pass
Staff API with admin token	Admin token on /api/staff/complaints	403 Staff access only	Pass
Assign to non-staff	Assign a report to a citizen ID	400 choose an approved staff member	Pass
Private documents	Open /uploads/... directly	404 Not Found	Pass
Pending staff login	Staff not yet approved logs in	403 waiting for admin approval	Pass

Test case	Input / action	Expected result	Result
Approval e-mail	Admin approves an applicant	E-mail sent to applicant	Pass

4.5 Version Control and Deployment Preparation

The source code is managed with Git and published to a GitHub repository. A *.gitignore* file ensures that secrets (*.env*), installed packages (*node_modules*) and private uploaded documents are never committed. The frontend can be built for production with *npm run build*, which produces optimised static files, and the backend is started with *npm start*.

4.6 Summary of Deliverables

- Working web application (frontend and backend source code).
- GitHub repository with README and environment template.
- Admin creation script for secure administrator accounts.
- This project report with architecture, data flow, use case and database diagrams.

5. Data Collection

SOLVEXA collects only the data needed to report, route and resolve civic issues and to verify staff. This chapter describes what is collected, how it is validated and how it is protected.

5.1 Data Collected from Citizens

Data item	Purpose	Required
Full name	Identify the reporter to administrators and staff	Yes
E-mail address	Login identity and password reset	Yes
Phone number	Contact by field staff; password reset verification	Yes
Password	Account security (stored only as a bcrypt hash)	Yes (not for Google sign-in)
Issue title and description	Explain the problem	Yes
Category	Route the issue to the right type of work	Yes
Priority	Low, Medium, High or Urgent	Yes (default Medium)
Location / address	Tell staff where the problem is	Yes

5.2 Data Collected from Staff Applicants

People applying to become field staff complete a five-step application. The data is used only by administrators to verify identity and suitability.

Step	Data collected
1. Account	Full name (as on Aadhaar), mobile number, e-mail, password
2. Personal	Date of birth, gender, full address, city, PIN code
3. Identity	Aadhaar number, PAN number, Aadhaar card image/PDF, PAN card image/PDF
4. Education & work	Highest qualification, institution, year of passing, years of experience, preferred department, skills, education certificate, resume (optional)
5. Photo & review	Passport-size photograph and a declaration that the information is true

5.3 System-Generated Data

- **Complaint number** in the format SOL-YYYY-NNNN, generated when a report is submitted.
- **Timestamps** (createdAt, updatedAt) on every user and complaint, used for "time ago" displays, charts and average resolution time.
- **Status history** through the status field and resolution details written by staff.
- **Assignment** — the staff member linked to each complaint.
- **Approval information** — application date, review date and rejection reason for staff.

5.4 Validation Rules

All important rules are checked twice: in the browser for instant feedback, and again on the server so they cannot be bypassed.

Field	Rule
E-mail	Must match name@domain format; stored in lower case; must be unique
Phone	10–13 digits, optional leading +
Password	At least 6 characters; strength meter guides users
Aadhaar	12 digits, first digit 2–9, valid Verhoeff check digit
PAN	Pattern AAAAA9999A (five letters, four digits, one letter)
Date of birth	Applicant age between 18 and 65 years
PIN code	Six digits, not starting with 0
Year of passing	Between 1960 and the current year
Uploads	JPG, PNG, WEBP (photo) or also PDF (documents); maximum 5 MB each
Status / priority	Must be one of the allowed values in the database schema

5.5 Data Privacy and Protection

- Passwords are hashed with bcrypt and never stored or displayed in plain text.
- Identity documents are saved with random file names in a private server folder that is not publicly served. They can only be downloaded through an administrator-only API that checks the login token.

- Aadhaar and PAN numbers are masked in the admin interface (for example XXXX XXXX 0124); administrators verify them against the uploaded card images.
- Staff only see the name and phone number of citizens on reports assigned to them.
- If an application fails validation, any files already uploaded are deleted immediately.
- Secrets are kept in environment variables and excluded from the Git repository.

5.6 Public (Anonymous) Statistics

The home page shows live figures — total issues reported, issues resolved, resolution rate, average time to resolve and number of citizens — together with a feed of recent activity. This data is produced by a public API that returns **only anonymous totals**: category, status and time, never names, titles, phone numbers or addresses.

6. Project Management

6.1 Development Methodology

The project followed an **iterative and incremental** approach inspired by Agile practices. Instead of building everything at once, the system was developed in short iterations. Each iteration delivered one working feature — for example login, then reporting, then the admin console — which was demonstrated, reviewed and improved based on feedback before the next feature began. This made it possible to change requirements (such as adding the staff application with document uploads) without disrupting completed work.

6.2 Project Phases and Schedule

Phase	Activities	Outcome
1. Planning	Problem study, user identification, feature list	Requirements and scope
2. Design	Architecture, database schema, API and UI design	Diagrams and data model
3. Core development	Authentication, reporting, citizen dashboard and tracking	Working citizen features
4. Administration	Admin login, console, charts, report management	Working admin console
5. Staff workflow	Staff approval, assignment, staff workspace	End-to-end resolution flow
6. Recruitment & e-mail	Job application, documents, review, e-mail	Verified staff onboarding
7. UI/UX refinement	Home page, registration redesign, animations, responsive layouts	Polished interface
8. Testing & release	Testing, security checks, documentation, GitHub	Release and this report

6.3 Tools Used for Managing the Project

- **Git and GitHub** — version history, backup and sharing of source code.
- **Visual Studio Code** — code editing, integrated terminal and file management.
- **npm** — managing third-party packages and running scripts (dev, build, lint, start).
- **ESLint** — enforcing consistent, error-free code in the frontend.

- **MongoDB Atlas dashboard** — database monitoring and network access control.
- **Google Cloud Console** — OAuth client configuration for Google sign-in.

6.4 Risk Management

Risk	Impact	Mitigation
Database unreachable (IP not allowed)	App cannot load or save data	Add IP to Atlas access list; clear error messages; server keeps running
Leak of secrets	Unauthorised access to database or e-mail	.env excluded from Git; .env.example template; App Passwords can be revoked
Misuse of staff role	Access to citizen contact details	Staff must apply and be approved by an admin before login
Fake identity details	Unverified workers	Checksum/pattern validation plus manual document review
Large or malicious uploads	Disk usage, unsafe files	5 MB limit, file-type whitelist, random file names, private folder
External service failure	No Google login or e-mail	E-mail/password login still works; admin is told when e-mail was not sent

6.5 Configuration and Secrets Management

All configuration that differs between environments or must stay secret is stored in a *server/.env* file, read at start-up by the *dotenv* package. The repository contains a *.env.example* file listing every setting with empty values:

```
PORT=5000
MONGO_URI=mongodb+srv://<username>:<password>@<cluster>.mongodb.net/solvexa
JWT_SECRET=<long random string>
GOOGLE_CLIENT_ID=<OAuth client id>.apps.googleusercontent.com
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=<sender gmail address>
SMTP_PASS=<gmail app password>
MAIL_FROM=SOLVEXA <sender gmail address>
CLIENT_URL=http://localhost:5173
```

7. System Architecture

7.1 Architectural Overview

SOLVEXA uses a **three-tier architecture**. The **presentation tier** is a React single-page application running in the user's browser. The **application tier** is a Node.js server using the Express framework, which exposes a REST API, checks permissions and applies business rules. The **data tier** consists of a MongoDB Atlas database and a private folder for uploaded documents. Two external services are used: Google Identity Services for sign-in and Gmail SMTP for e-mail.

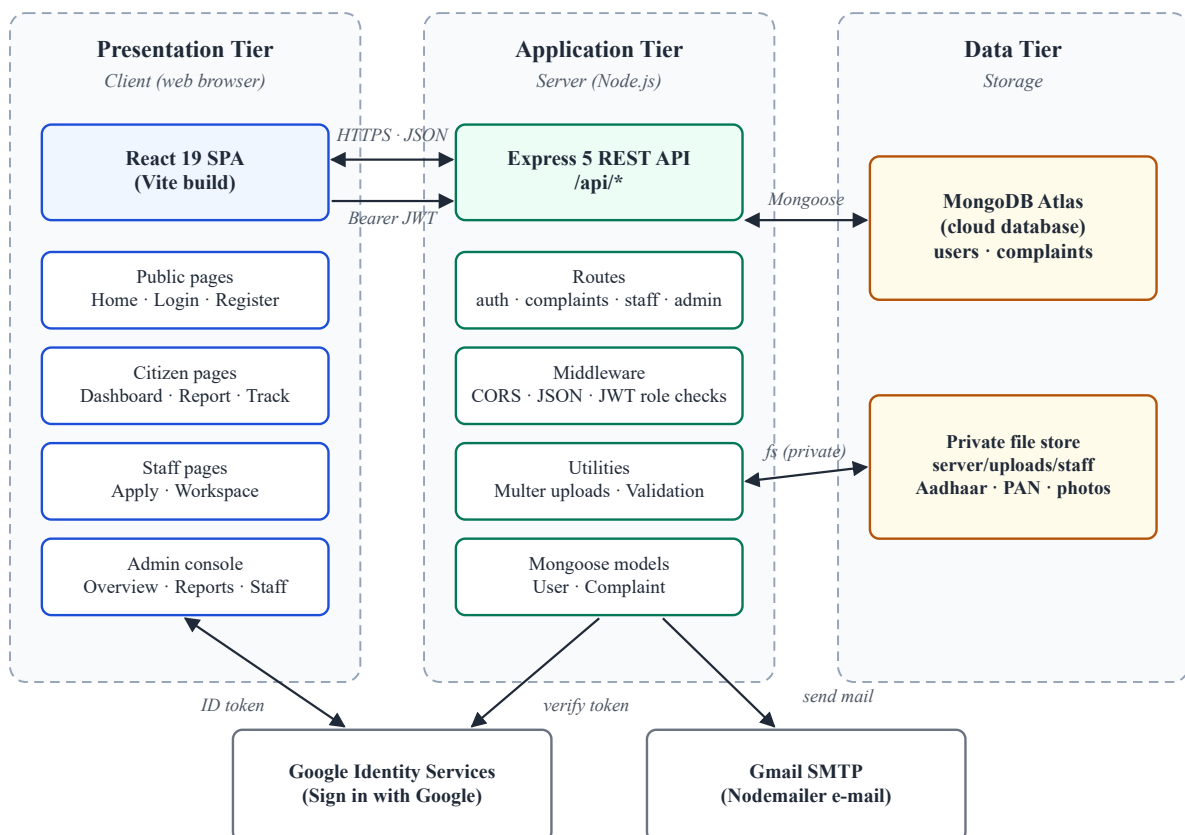


Figure 7.1: Three-tier architecture of SOLVEXA

A typical request works as follows: the user performs an action in the browser; the React page sends an HTTP request with JSON data (and a JWT token for protected actions) to the API; Express routes the request, middleware verifies the token and role, the route handler reads or writes MongoDB through Mongoose models, and a JSON response is returned and displayed.

7.2 Data Flow Diagram – Level 0

The Level 0 (context) diagram shows SOLVEXA as a single process and the external entities that exchange data with it.

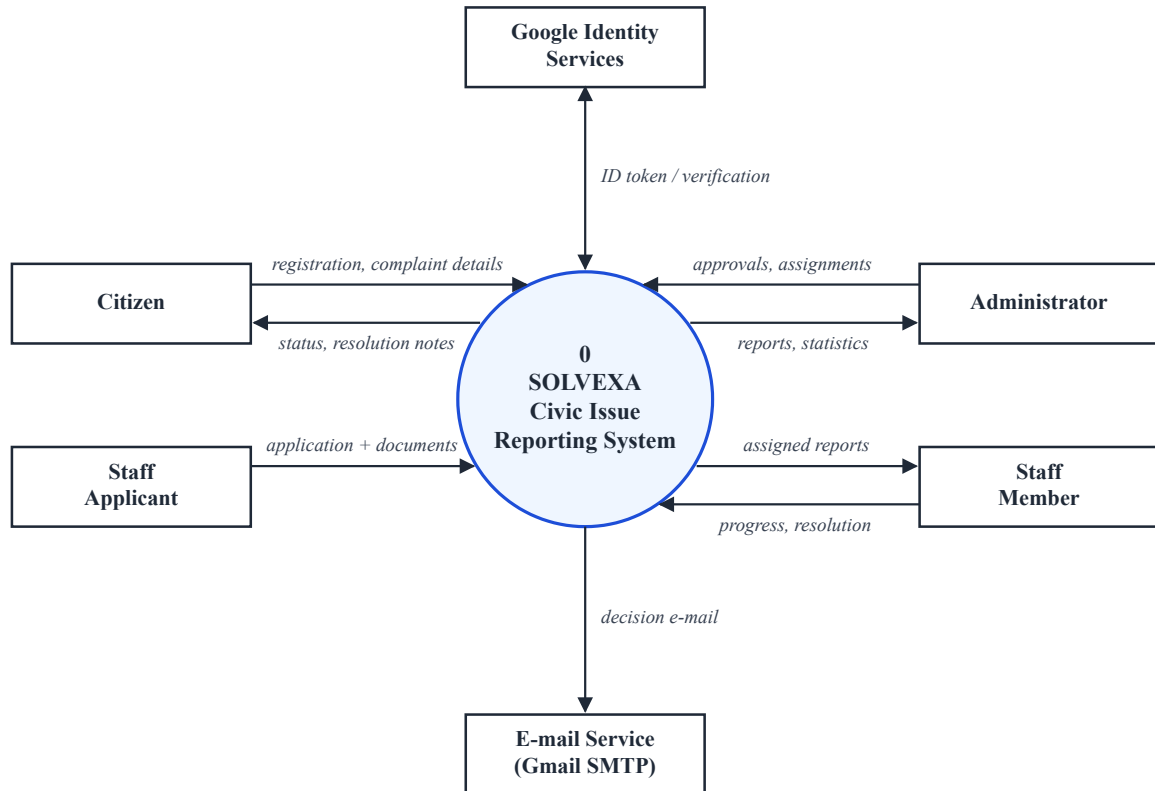


Figure 7.2: DFD Level 0 – context diagram

7.3 Data Flow Diagram – Level 1

The Level 1 diagram breaks the system into its six main processes and shows the data stores they use: D1 Users, D2 Complaints and D3 Document Files.

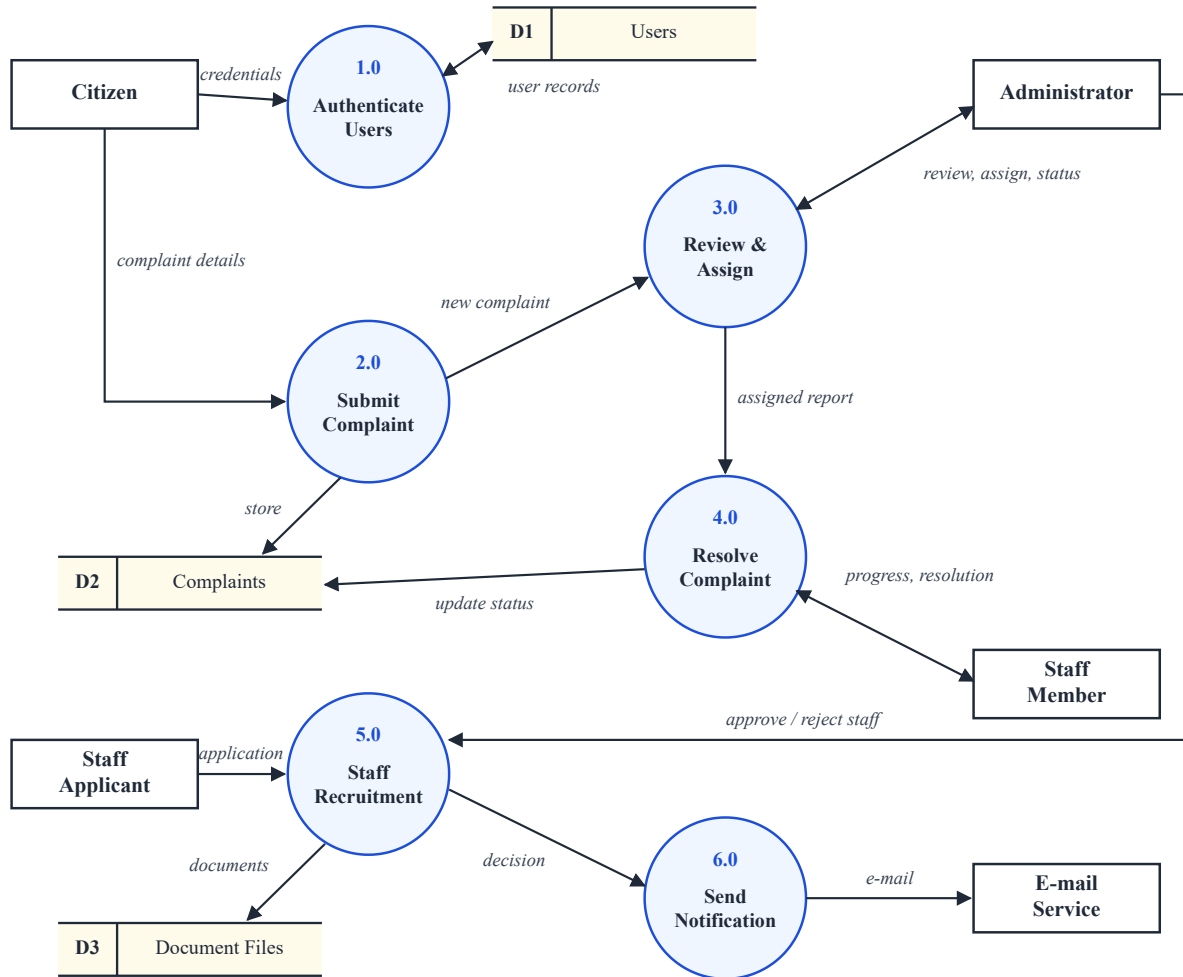


Figure 7.3: DFD Level 1 – main processes and data stores

Process	Description
1.0 Authenticate Users	Registers users, verifies passwords or Google tokens, issues JWT tokens and enforces staff approval.
2.0 Submit Complaint	Validates a new report, generates the complaint number and stores it in D2.
3.0 Review & Assign	Lets the administrator review reports, set priority and status, and assign staff.
4.0 Resolve Complaint	Lets assigned staff start work and record resolution details.

Process	Description
5.0 Staff Recruitment	Validates applications, stores documents in D3 and records admin decisions.
6.0 Send Notification	Sends approval or rejection e-mails to applicants.

7.4 Use Case Diagram

The use case diagram shows the four actors and the main goals each can achieve with the system.

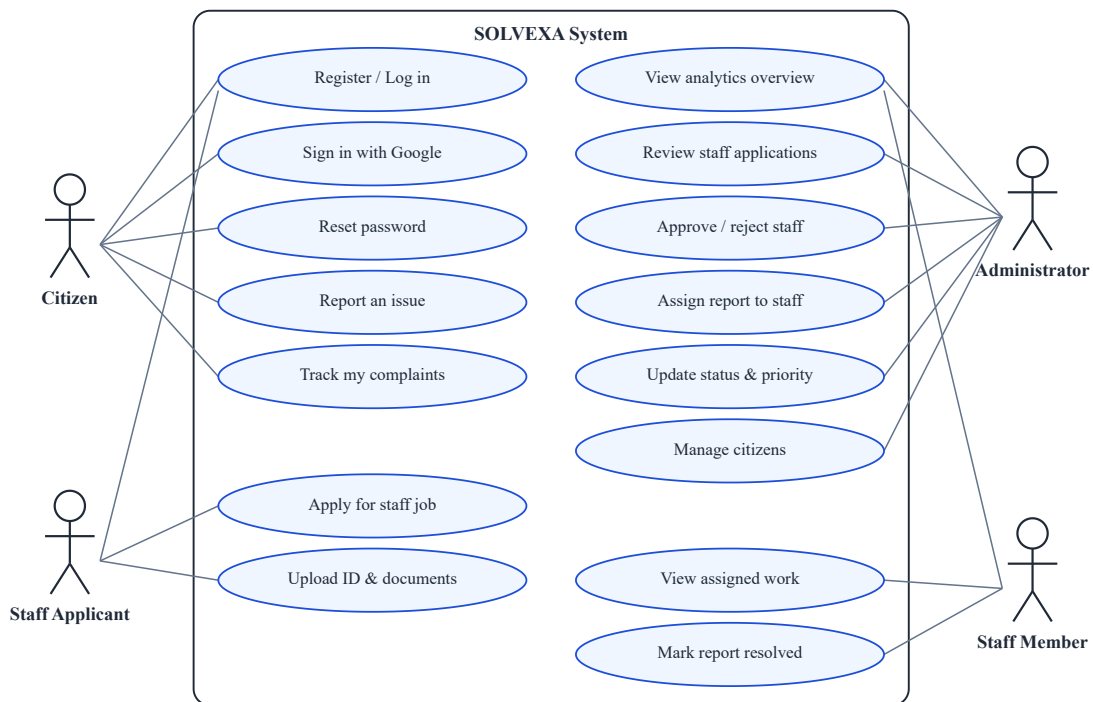


Figure 7.4: Use case diagram

7.5 Complaint Lifecycle

Every complaint moves through six states. The administrator reviews and assigns; the staff member starts work and resolves; the administrator finally closes the report. If a problem was not really fixed, staff can reopen a resolved report back to In Progress.

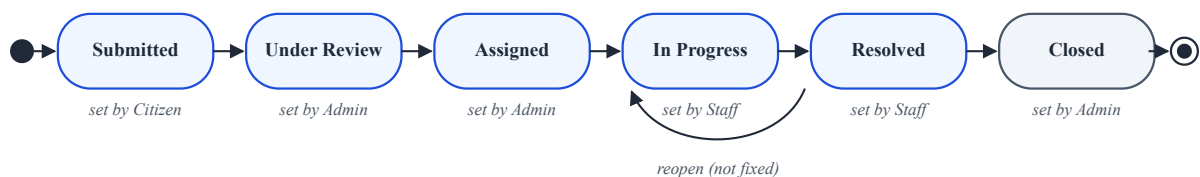


Figure 7.5: Complaint status lifecycle

7.6 Database Schema

SOLVEXA stores its data in two MongoDB collections, *users* and *complaints*, defined with Mongoose schemas. The relationship between them is shown below.

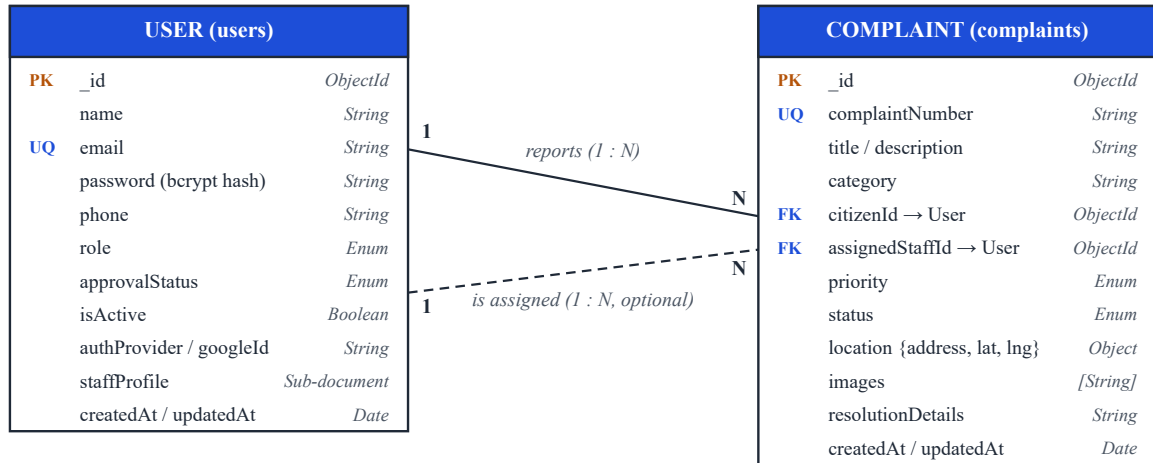


Figure 7.6: Entity–relationship diagram

Collection: users

Field	Type	Constraints / description
<code>_id</code>	ObjectId	Primary key (generated)
<code>name</code>	String	Required, trimmed
<code>email</code>	String	Required, unique, lower case
<code>password</code>	String	Required; bcrypt hash
<code>phone</code>	String	Required
<code>role</code>	String	citizen staff department admin (default citizen)
<code>approvalStatus</code>	String	pending approved rejected (staff approval)
<code>isActive</code>	Boolean	Default true; inactive users cannot log in
<code>departmentId</code>	ObjectId	Reserved for future department support
<code>profileImage</code>	String	Optional (e.g. Google profile picture)
<code>authProvider</code>	String	email google
<code>googleId</code>	String	Google account identifier when linked

Field	Type	Constraints / description
staffProfile	Object	Job application: dateOfBirth, gender, address, city, pincode, aadhaarNumber, panNumber, qualification, institution, yearOfPassing, experienceYears, preferredDepartment, skills, documents {photo, aadhaarCard, panCard, educationCertificate, resume}, appliedAt, reviewedAt, rejectionReason
createdAt, updatedAt	Date	Automatic timestamps

Collection: complaints

Field	Type	Constraints / description
_id	ObjectId	Primary key (generated)
complaintNumber	String	Required, unique, e.g. SOL-2026-0001
title	String	Required, trimmed
description	String	Required, trimmed
category	String	Required (one of eight categories)
citizenId	ObjectId → users	Required; the reporting citizen
assignedStaffId	ObjectId → users	Staff member assigned (default null)
departmentId	ObjectId	Reserved for future department support
priority	String	Low Medium High Urgent (default Medium)
status	String	Submitted Under Review Assigned In Progress Resolved Closed
images	[String]	Reserved for issue photos (default empty)
location	Object	address (String), latitude, longitude (Number)
resolutionDetails	String	Note to citizen describing the action taken
createdAt, updatedAt	Date	Automatic timestamps

7.7 REST API Design

The backend exposes the following endpoints. Endpoints under */api/admin* require an administrator token and endpoints under */api/staff* require an approved staff token.

Method	Endpoint	Purpose
POST	/api/auth/register	Register a citizen

Method	Endpoint	Purpose
POST	/api/auth/login	Log in (citizen or staff) and receive a JWT
POST	/api/auth/google	Sign up / log in with a Google ID token
GET	/api/auth/google-config	Public Google Client ID for the sign-in button
POST	/api/auth/forgot-password	Reset password with e-mail and phone
POST	/api/auth/admin-login	Administrator login
GET	/api/auth/staff-application/options	Lists of qualifications, departments, genders
POST	/api/auth/register-staff	Submit staff application with documents (multipart)
POST	/api/complaints	Create a complaint
GET	/api/complaints/citizen/:citizenId	A citizen's complaints
GET	/api/complaints/stats/:citizenId	A citizen's complaint counts
GET	/api/complaints/public/summary	Anonymous public statistics
GET	/api/complaints/:id	A single complaint
GET	/api/admin/stats	Overall statistics
GET	/api/admin/complaints	All complaints with citizen and staff details
PUT	/api/admin/complaints/:id	Update status, priority, notes, assignment
GET	/api/admin/citizens	Citizens with complaint counts
GET	/api/admin/staff	Staff with workload and (masked) application
PUT	/api/admin/staff/:id/approval	Approve or reject staff and send e-mail
POST	/api/admin/staff/:id/notify	Re-send the decision e-mail
GET	/api/admin/staff/:id/documents/:document	Download a private applicant document
GET	/api/admin/email-status	Whether e-mail is configured
GET	/api/staff/complaints	Reports assigned to the logged-in staff member
PUT	/api/staff/complaints/:id	Staff progress update (In Progress / Resolved)

7.8 Security Architecture

- **Authentication:** JWT tokens signed with a secret key and valid for one day.
- **Authorisation:** middleware re-reads the user from the database on each admin or staff request, so removing a role or approval takes effect immediately.
- **Password storage:** bcrypt hashing with a cost factor of 10.
- **Google sign-in:** the server verifies the Google ID token (issuer, audience and verified e-mail) before trusting it.
- **Role creation:** the public register form can only create citizens; staff must apply and be approved; administrators are created only with a server-side script.
- **File security:** type and size limits, random names, private storage and path sanitisation when documents are served.
- **CORS:** the API only accepts browser requests from the SOLVEXA frontend origin.

8. External Interface Requirements

8.1 User Interfaces

The interface is designed to be simple, consistent and responsive, adapting to phone, tablet and desktop screens. Its main screens are:

Screen	Description
Home	Animated hero illustration, live statistics, how-it-works steps, categories, live activity, FAQ and emergency numbers.
Register	Split-screen design with Google sign-up, e-mail sign-up, live validation and password strength meter.
Login	Citizen / Staff switch, Google login, forgot password link and admin login link.
Citizen dashboard	Welcome header, complaint statistics, quick actions and recent complaints.
Report an issue	Form for title, category, priority, description and location.
My Complaints	Cards for each complaint with a six-step progress tracker and handling staff.
Staff application	Five-step form with progress bar, file uploads with previews and review page.
Staff workspace	Assigned reports sorted by urgency with Start work and Mark as resolved actions.
Admin console	Sidebar navigation: Overview (KPIs and charts), Reports, Staff and Citizens.

8.2 Hardware Interfaces

SOLVEXA has no special hardware requirements. Users need any device capable of running a modern web browser. The server can run on a standard computer or cloud virtual machine with at least 2 GB of RAM. Documents are uploaded from the device's file system or camera gallery.

8.3 Software Interfaces

Software	Interface
MongoDB Atlas	Database accessed through the Mongoose ODM over a secure connection string
Google Identity Services	JavaScript library in the browser; token verified via Google's tokeninfo endpoint
Gmail SMTP	E-mail sent with Nodemailer on port 587 using STARTTLS and an App Password
Web browser	Runs the React application; uses local storage for the session token

8.4 Communications Interfaces

The browser and server communicate over HTTP(S) using JSON, except for staff applications, which use multipart/form-data to carry files. Protected requests include an *Authorization: Bearer <token>* header. The server communicates with MongoDB over TLS and with Gmail over SMTP with STARTTLS. In production, all traffic should be served over HTTPS.

9. System Features

9.1 Citizen Registration and Authentication

Description and priority: Allows people to create accounts and log in securely. Priority: High.

Stimulus/response: The user fills in the registration form or clicks "Sign up with Google"; the system validates the data, creates the account and redirects to login or the dashboard.

- REQ-1: The system shall register citizens with name, e-mail, phone and password.
- REQ-2: The system shall reject duplicate e-mail addresses.
- REQ-3: The system shall allow sign-up and login with a verified Google account.
- REQ-4: The system shall issue a JWT token valid for one day after login.
- REQ-5: The system shall let users reset their password using their e-mail and registered phone.
- REQ-6: The system shall always open the home page after logout or page reload.

9.2 Issue Reporting and Tracking

Description and priority: The core feature — citizens report and follow issues. Priority: High.

- REQ-7: The system shall let a logged-in citizen submit an issue with title, category, priority, description and location.
- REQ-8: The system shall generate a unique complaint number for each issue.
- REQ-9: The system shall show each citizen only their own complaints with a six-step progress tracker.
- REQ-10: The system shall show the name of the staff member handling a complaint and the resolution notes.

9.3 Administrator Console

Description and priority: Management interface for administrators. Priority: High.

- REQ-11: The system shall provide separate administrator login.
- REQ-12: The system shall display statistics: total, open, urgent, unassigned, resolution rate, citizens.

- REQ-13: The system shall display charts of reports per day, status pipeline and reports by category.
- REQ-14: The system shall allow searching and filtering reports by status, category, priority and assignee.
- REQ-15: The system shall allow administrators to change status and priority and add notes.

9.4 Staff Assignment and Workspace

Description and priority: Connects reports to the people who fix them. Priority: High.

- REQ-16: The system shall allow administrators to assign a report to an approved staff member, showing each member's open workload.
- REQ-17: Assigning a Submitted or Under Review report shall automatically set its status to Assigned.
- REQ-18: Staff shall see only reports assigned to them, ordered by priority.
- REQ-19: Staff shall be able to start work (In Progress) and resolve a report with a mandatory note.
- REQ-20: Only administrators shall be able to close a report.

9.5 Staff Recruitment and E-mail Notification

Description and priority: Verified onboarding of field staff. Priority: Medium.

- REQ-21: The system shall provide a five-step staff application collecting personal, identity, education details and documents.
- REQ-22: The system shall validate Aadhaar (Verhoeff checksum) and PAN (pattern) numbers.
- REQ-23: The system shall prevent unapproved or rejected staff from logging in.
- REQ-24: The system shall allow administrators to view application details and documents, and approve or reject with a reason.
- REQ-25: The system shall e-mail the applicant the decision and allow re-sending the e-mail.

10. Technologies Used

10.1 Programming Languages

Language	Where it is used
JavaScript (ES2022+)	The entire application — frontend logic and backend server
JSX	React component markup in the frontend
HTML5	Page structure (index.html) and rendered components
CSS3	All styling, responsive layouts and animations (custom CSS, no framework)
SVG	Illustrations and icons such as the animated city scene
JSON	Data exchange between frontend and backend; configuration files

10.2 Frontend Technologies

Technology	Version	Purpose
React	19.2	Component-based user interface library
Vite	8.3	Development server and production build tool
React Router DOM	7.18	Page routing (Home, Login, Dashboard, Admin, ...)
Axios	1.20	HTTP requests to the backend API
Recharts	3.10	Charts in the administrator overview
Leaflet / React-Leaflet	1.9 / 5.0	Included for a future map view of issues
ESLint	10.10	Code quality checks (React hooks rules)

10.3 Backend Technologies

Technology	Version	Purpose
Node.js	24	JavaScript runtime for the server
Express	5.2	Web framework for the REST API
Mongoose	9.10	MongoDB schemas, validation and queries
jsonwebtoken	9.0	Creating and verifying login tokens

Technology	Version	Purpose
bcryptjs	3.0	Password hashing
Multer	2.3	Handling file uploads for staff applications
Nodemailer	10.0	Sending e-mail through Gmail SMTP
dotenv	17.4	Loading configuration from the .env file
cors	2.8	Allowing requests from the frontend origin

10.4 Database

MongoDB Atlas is a fully managed cloud version of MongoDB, a NoSQL document database. Data is stored as flexible JSON-like documents, which fits naturally with JavaScript on both frontend and backend. Nested data such as a complaint's location or a staff member's application is stored inside the same document, avoiding complex joins. Atlas provides automatic backups, security features such as IP access lists, and a free tier suitable for development.

10.5 Development Tools and Services

Tool / service	Use in the project
Visual Studio Code	Code editor with integrated terminal
Git & GitHub	Version control and online code repository
npm	Package management and scripts
Google Chrome DevTools	Debugging and responsive-design testing
Playwright (headless Chrome)	Automated UI checks and screenshots during development
MongoDB Atlas	Cloud database hosting and monitoring
Google Cloud Console	OAuth client for Google sign-in
Gmail (App Password)	SMTP account for sending notification e-mails

11. Other Nonfunctional Requirements

11.1 Performance Requirements

- Pages should become interactive within about 2 seconds on a normal broadband connection.
- Common API requests should respond within 1 second under normal load.
- The admin console loads statistics, reports, citizens and staff in parallel to reduce waiting time.
- Database queries use indexed fields such as unique e-mail and complaint number.

11.2 Safety Requirements

- SOLVEXA is not an emergency service. The home page clearly directs emergencies to 112 (Emergency), 108 (Ambulance) and 101 (Fire).
- Actions that remove staff access ask for confirmation, and rejections require a reason.
- Failed uploads are cleaned up automatically so the server is not filled with partial data.

11.3 Security Requirements

- All passwords shall be hashed; no password shall be logged or displayed.
- Administrator and staff APIs shall reject requests without a valid token of the correct role.
- Identity documents shall never be publicly accessible.
- Secrets shall never be committed to version control.
- The system should be deployed behind HTTPS in production.

11.4 Software Quality Attributes

Attribute	How SOLVEXA achieves it
Usability	Clear layouts, helpful messages, live validation, step-by-step forms and visual progress trackers
Accessibility	Readable fonts, labelled form fields, keyboard support (e.g. Escape closes panels), reduced motion for users who prefer it
Responsiveness	Layouts adapt to phones, tablets and desktops
Maintainability	Modular components and routes, consistent naming, ESLint checks, README and environment template

Attribute	How SOLVEXA achieves it
Reliability	Server-side validation, graceful error messages, database connection checks
Portability	Runs on any OS with Node.js and any modern browser
Scalability	Stateless API with token authentication can be scaled across servers; cloud database

11.5 Business Rules

- Only citizens can register through the public form; staff must apply; admins are created by script.
- Staff can log in only after an administrator approves their application.
- A report can only be assigned to an approved, active staff member.
- Staff may set status to Assigned, In Progress or Resolved; only administrators may close a report.
- A report cannot be marked Resolved by staff without a resolution note.
- Staff access cannot be removed while they still have open assigned reports.

12. Implementation and Example Walkthrough

12.1 Installation and Setup

Prerequisites: Node.js (version 20 or later), Git, a MongoDB Atlas account, and optionally a Google Cloud OAuth client and a Gmail App Password.

```
# 1. Get the source code
git clone https://github.com/pranathisubbarao-svg/SOLVEXA.git
cd SOLVEXA

# 2. Install backend and frontend packages
cd server
npm install
cd ../client
npm install

# 3. Configure the backend
#   copy server/.env.example to server/.env and fill in the values

# 4. Create the first administrator
cd ../server
npm run create-admin -- admin@example.com Admin@123 "Admin" 9876543210
```

12.2 Running the Application

Open two terminals — one for the backend and one for the frontend:

```
# Terminal 1 - backend API on http://localhost:5000
cd SOLVEXA/server
npm run dev

# Terminal 2 - website on http://localhost:5173
cd SOLVEXA/client
npm run dev
```

Then open `http://localhost:5173` in a browser. For production, run `npm run build` in the client folder to create optimised files and `npm start` in the server folder.

12.3 Example: From Report to Resolution

The following example shows how the system works end to end.

Step	Who	Action in SOLVEXA	Result
1	Ravi (applicant)	Applies as staff for the Water Supply department and uploads his documents	Application appears under New staff applications
2	Admin	Reviews Ravi's details and documents and clicks Approve	Ravi receives a "You're selected" e-mail and can log in
3	Ananya (citizen)	Registers and reports "Large pothole near main road" with priority High	Complaint SOL-2026-0001 created with status Submitted
4	Admin	Opens the report and assigns it to Ravi	Status becomes Assigned; appears in Ravi's workspace
5	Ravi (staff)	Clicks Start work	Status In Progress; Ananya sees the progress
6	Ravi (staff)	Fixes the pothole, writes "Pothole filled and road levelled", marks resolved	Status Resolved; note visible to Ananya
7	Admin	Verifies the work and closes the report	Status Closed; statistics and home page update

12.4 Sample API Request and Response

Creating a complaint (step 3 above) sends the following request to the server:

```
POST /api/complaints
Content-Type: application/json

{
  "title": "Large pothole near main road",
  "description": "A large pothole is causing difficulty for vehicles and pedestrians.",
  "category": "Roads & Potholes",
  "priority": "High",
  "citizenId": "6a1f...c21",
  "location": { "address": "Near main road, Hyderabad", "latitude": null, "longitude": null }
}
```

The server responds with the saved complaint, including its generated number and initial status:

201 Created

```
{
  "message": "Complaint submitted successfully",
  "complaint": {
    "_id": "6a2b...9f0",
    "complaintNumber": "SOL-2026-0001",
    "status": "Submitted",
    "priority": "High",
    "assignedStaffId": null,
    "createdAt": "2026-09-22T00:23:38.623Z",
    ...
  }
}
```

13. Conclusion and Future Enhancements

13.1 Conclusion

SOLVEXA successfully demonstrates how a modern web application can make civic problem reporting simple, transparent and accountable. Citizens can report an issue in under a minute and follow it through every stage; administrators have a complete picture of all reports with charts and filters and can assign work in one click; and field staff have a focused workspace for their tasks. The integrated staff recruitment process, with identity validation, private document review and e-mail notifications, ensures that only verified people handle citizens' information.

The project applied the MERN stack, REST API design, token-based security, third-party integrations and responsive UI design to a real social problem, and it is ready to be extended and deployed for use by a local community.

13.2 Limitations

- Citizens cannot yet attach photos to complaints (the database field exists but the upload is not built).
- Location is entered as text; there is no map pin yet.
- Citizens are not yet notified by e-mail or SMS when their complaint status changes.
- Citizen complaint endpoints identify the user by ID from the browser; they should be protected with the same token checks used for admin and staff endpoints.
- Departments are reserved in the schema but not yet managed in the interface.

13.3 Future Enhancements

- **Interactive maps** using Leaflet to pin issues and show hotspots.
- **Photo uploads** for complaints, and before/after photos from staff.
- **Notifications** by e-mail, SMS or push when status changes.
- **Department management** with automatic routing by category.
- **Mobile app** (for example with React Native) and offline reporting.
- **Multi-language support** such as Telugu and Hindi.
- **Citizen feedback and ratings** after resolution.
- **Cloud deployment** with HTTPS, a custom domain and monitoring.
- **Analytics** such as average resolution time per department and category trends.

14. References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.
2. React documentation — <https://react.dev>
3. Vite documentation — <https://vite.dev>
4. React Router documentation — <https://reactrouter.com>
5. Node.js documentation — <https://nodejs.org/docs>
6. Express documentation — <https://expressjs.com>
7. MongoDB documentation — <https://www.mongodb.com/docs>
8. Mongoose documentation — <https://mongoosejs.com/docs>
9. JSON Web Tokens (RFC 7519) — <https://datatracker.ietf.org/doc/html/rfc7519>
10. Google Identity Services documentation — <https://developers.google.com/identity/gsi/web>
11. Nodemailer documentation — <https://nodemailer.com>
12. Multer documentation — <https://github.com/expressjs/multer>
13. Recharts documentation — <https://recharts.org>
14. Verhoeff, J. (1969). *Error Detecting Decimal Codes*. Mathematical Centre Tracts 29, Amsterdam.
15. Centralised Public Grievance Redress and Monitoring System (CPGRAMS) — <https://pgportal.gov.in>

Appendix A. Glossary

Term	Definition
Aadhaar	A 12-digit unique identity number issued to residents of India
PAN	Permanent Account Number, a 10-character identifier issued by the Income Tax Department of India
API	Application Programming Interface — the set of endpoints the frontend uses to talk to the server
CORS	Cross-Origin Resource Sharing — browser rules about which websites may call an API
CRUD	Create, Read, Update, Delete — the basic data operations
DFD	Data Flow Diagram — shows how data moves between processes, stores and external entities
JWT	JSON Web Token — signed token proving a user's identity and role
KPI	Key Performance Indicator — a headline number such as resolution rate
MERN	MongoDB, Express, React, Node.js
ODM	Object–Document Mapper (Mongoose)
SMTP	Simple Mail Transfer Protocol — used to send e-mail
SPA	Single-Page Application